

RelevanceEngine

Build a Search Engine From Scratch, Then a RAG System — Measured, Not Vibed

Before You Start: Pre-Search (30 Minutes)

Before writing any code, complete the Pre-Search methodology at the end of this document. This structured process uses AI to explore stack options, surface tradeoffs, and document architecture decisions. Your Pre-Search output becomes part of your final submission.

AI-first development is central to this assignment, and Pre-Search is the first step in that methodology.

Background

Retrieval and RAG are what most AI product teams are actually building — and the field’s defining failure is bolting an LLM onto bad retrieval and never noticing. This project makes you earn good retrieval first. You build an **inverted index** over a real corpus with **BM25 ranked retrieval**, and you measure its quality objectively against a labeled relevance set — precision@k, recall, MRR, NDCG — before an LLM is anywhere near the system.

Then, in the Extension, you turn it into a **retrieval-augmented generation** system: embeddings for semantic search, an LLM that answers grounded in the retrieved documents, with citations, and honest evals of both retrieval recall and answer faithfulness. “It returns ten results” feels like success and means nothing; a fluent answer that cites the wrong chunk is the #1 RAG failure. The whole point is to learn what “good retrieval” means and how to measure it — the eval discipline here transfers straight to your first role.

Project Overview

A five-day sprint with three checkpoints:

Checkpoint	Deadline	Focus
MVP	Tuesday at 11:59 PM	Inverted index + boolean retrieval + a red metrics harness on a labeled set
Early Submission	Thursday at 11:59 PM	BM25 ranking, full IR-metrics harness, and a reference-BM25 differential test
Final	Sunday at 11:59 AM	Tuning against metrics, robustness, a metrics report, and a self-eval report

MVP Requirements (Day 1 — 24 Hours)

Day 1 baseline. Aim to have all of these working before you build outward:

- ☐ Day-1 design doc committed (analysis pipeline, index representation, ranking, judgment set)
- ☐ Corpus chosen and loaded (a few thousand documents)
- ☐ A hand-built relevance-judgment set started (≥ 5 queries with labeled relevant docs)
- ☐ Inverted index built with document frequencies and term positions
- ☐ Boolean retrieval (`AND / OR`) returning documents end-to-end
- ☐ Metrics harness stubbed with `precision@k` / `recall` on the labeled queries (red)
- ☐ Runnable from a clean clone with one command

Without a judgment set you cannot tell good retrieval from bad. Build the labels first.

Core Search Engine

A search service (CLI or a tiny HTTP endpoint) over your chosen corpus — a Wikipedia subset, a docs set, or a papers dump. Retrieval quality is objective here; treat the metrics as your daily scoreboard.

Indexing & Retrieval

Feature	Requirements
Analysis	Tokenize + normalize (case, punctuation, optional stemming/stopwords) — a documented choice
Inverted index	Term $\rightarrow$ sorted doc-id postings with document frequencies and positions
Ranking	BM25 with defended <code>k1</code> and <code>b</code> ; top-k with scores
Boolean ops	<code>AND / OR</code> , and ideally phrase / proximity queries
Snippets	Top-k results returned with scores and highlighted snippets

Evaluation Set

Feature	Requirements
Judgment set	≥ 15 queries with labeled relevant docs, built without fooling yourself
IR metrics	<code>precision@k</code> , <code>recall@k</code> , <code>MRR</code> , <code>NDCG</code> against the labeled set — your primary score
Differential	Scores/ranking match a reference BM25 within tolerance on the same corpus
Robustness	Empty queries, stopword-only queries, unicode, absent terms — no crash or corruption

Testing Scenarios

We will test:

- 1. IR metrics against your labeled query→relevant-docs set — precision@k, recall@k, MRR, NDCG tracked every day.
- 2. Differential vs. a well-known BM25 library on the same corpus and query — scores/ranking match within tolerance.
- 3. Property checks: IDF decreases as a term appears in more documents; a doc gaining a query term cannot rank below an identical doc lacking it.
- 4. Index-then-search round-trips every document; deleting a doc removes it from all its postings.
- 5. Robustness: empty queries, stopword-only queries, unicode, very long documents, terms absent from the corpus.
- 6. (Extension) Recall@k of the *right* chunks and answer faithfulness on a labeled QA set.

Correctness & Performance Targets

Metric	Target
Retrieval quality	Strong, honestly-measured precision/recall/NDCG on a real judgment set
Differential vs. reference BM25	Scores/ranking match within tolerance
Property invariants	IDF monotonicity + round-trip identity hold universally
Robustness	0 crashes on empty/unicode/edge queries; index stays stable
(Extension) Faithfulness	Every answer claim supported by retrieved context

RAG Extension — Grounded Answers With Citations

Question answering grounded in the corpus. Embed documents (or chunks) for semantic retrieval, retrieve top-k, pass them to an LLM, and return an answer **with citations to the source documents**. Ideally offer hybrid retrieval (BM25 lexical + embedding similarity) and compare. Support at least 4 distinct behaviours.

Required Capabilities

Grounded Q&A

“What problem does the Raft paper solve?” → answer + citation to the source chunk(s)

Hybrid retrieval: combine lexical BM25 scores with embedding similarity and compare

Refusal

A question the corpus cannot answer → the system declines instead of hallucinating

Citation

Every claim points to a chunk that actually supports it

Interface (Minimum)

```
retrieve(query, k, mode) -> chunks // mode: lexical | semantic | hybrid
answer(query, chunks) -> { text, citations[] }
faithful(answer, chunks) -> judge_verdict // every claim supported by context?
// evaluate retrieval recall AND answer faithfulness — fluency is not grounding
```

Evaluation Criteria

Dimension	How It’s Scored
-----------	-----------------

Retrieval quality	recall@k of the right chunks; compare lexical vs. semantic vs. hybrid on the same queries
Faithfulness / groundedness	LLM-judge rubric: every claim supported by retrieved context (the central RAG metric)
Answer relevancy & citations	Answer addresses the question; citations point to chunks that actually support the claims
Refusal behaviour	On an unanswerable-question set, how often the system correctly declines

Shared AI State & Discipline

- Chunking is a tuned parameter measured against retrieval metrics — not a default that “seems fine.”
- Faithfulness evals are non-negotiable: an ungrounded-but-fluent answer is the #1 RAG failure.
- Validate your LLM judge against ~10 hand labels; use a judge from a different model family than the generator.

AI Feature Performance

Metric	Target
Retrieval recall@k	Reported for lexical, semantic, and hybrid
Faithfulness	Honestly measured; ungrounded answers surfaced, not hidden
Refusal on unanswerable set	High and honestly reported

AI-First Development Requirements

This assignment emphasizes AI-first development workflows, so document your process as you go.

Required Tools

Use at least two of:

- Claude Code
- Cursor
- Codex
- MCP integrations

AI Development Log (Required)

Submit a 1-page document covering:

Section	Content
Tools & Workflow	Which AI coding tools you used, and how you integrated them into the build.
MCP Usage	Which MCPs you used (if any) and what they enabled.
Effective Prompts	3–5 prompts that worked well (include the actual prompt text).
Code Analysis	Rough % of AI-generated vs. hand-written code.
Strengths & Limitations	Where AI excelled, and where it struggled.
Oracle Catches	Places the assistant was confidently wrong and how the oracle / tests caught it.
Key Learnings	Insights about working with coding agents on a correctness-critical build.

AI Cost Analysis (Required)

Understanding AI costs is critical for production applications. Track and report your actual spend during development: LLM API costs, total tokens (input/output breakdown), number of API calls, and any other AI-related costs (embeddings, hosting).

Then estimate monthly costs at different user scales:

100 Users	1,000 Users	10,000 Users	100,000 Users
\$___/month	\$___/month	\$___/month	\$___/month

Include assumptions: average AI questions per user per session, sessions per user per month, and token counts per question type.

Technical Stack

Recommended Path

Layer	Technology
Language / Runtime	Any language you know well for the index + retrieval loop
Corpus	A Wikipedia subset, a docs set, or a papers dump (a few thousand docs)
Oracle	Labeled relevance judgments (IR metrics) + a reference BM25 implementation
Embeddings	Any embedding API/model for the semantic + hybrid Extension
AI Integration	OpenAI GPT-4-class or Anthropic Claude for the generation step

Alternative Options

Layer	Alternatives
Index storage	In-memory, on-disk, or memory-mapped postings
Analysis	Stemming/stopwords on or off — a documented, measured choice
Vector store	Brute-force cosine or a lightweight ANN index for the Extension

Use whatever stack helps you ship. Complete the Pre-Search process to make informed decisions.

Build Strategy

Priority Order

1. Judgment set + red metrics harness — labels before ranking
2. Inverted index + boolean retrieval — documents come back

- 3. BM25 + full metrics + reference-BM25 differential — now you can measure every change
- 4. Tune analysis and parameters using the metrics as your guide; add property/robustness tests
- 5. RAG: embeddings + semantic/hybrid retrieval + grounded answers with citations
- 6. Faithfulness + refusal evals — the guardrail that catches fluent-but-wrong

Critical Guidance

“It returns ten results” means nothing without the judgment set. Build the labels first.

A BM25 formula that is *almost* right (wrong IDF variant, missing length normalization) only shows up in the differential test and metrics.

In RAG, the model produces authoritative answers that cite the wrong chunk or invent facts — faithfulness evals are the catch. The AI will pick a chunk size that “seems fine” and quietly tank recall. Measure it.

Submission Requirements

Deadline: End of Day 5

Deliverable	Requirements
GitHub Repository	Setup guide, architecture overview, and (for web products) a deployed link.
Demo Video (3–5 min)	Show the core working against its oracle, the AI feature, and an architecture explanation.
Pre-Search Document	Completed checklist from Phase 1–3.
AI Development Log	1-page breakdown using the template above.
AI Cost Analysis	Dev spend + projections for 100 / 1K / 10K / 100K users.
Evaluation Harness	The relevance-judgment set + IR-metrics harness (precision/recall/MRR/NDCG) + differential + property + robustness, one command.
Metrics Report	A reproducible run of your metrics, in the README.
Self-Eval Report	Your harness pass rates against the rubric.
RAG Pipeline	The QA eval set + faithfulness/relevancy/refusal eval scripts, with reported numbers.
AI-LOG.md	Where AI misled you and how the oracle caught it.
Social Post	Share on X or LinkedIn: description, features, demo/screenshots. Tag @GauntletAI.

Final Note

Appendix: Pre-Search Checklist

Complete this before writing code. Save your AI conversation as a reference document.

Phase 1: Define Your Constraints

1. Scale & Load Profile

Users at launch? In 6 months?
Traffic pattern: steady, spiky, or unpredictable?
Real-time or batch requirements?
Cold-start tolerance?

2. Budget & Cost Ceiling

Monthly spend limit?
Pay-per-use acceptable or need fixed costs?
Where will you trade money for time?

3. Time to Ship

MVP timeline?
Speed-to-market vs. long-term maintainability priority?
Iteration cadence after launch?

4. Compliance & Regulatory Needs

Health data (HIPAA)?
EU users (GDPR)?
Enterprise clients (SOC 2)?
Data residency requirements?

5. Team & Skill Constraints

Solo or team?
Languages/frameworks you know well?
Learning appetite vs. shipping speed preference?

Phase 2: Architecture Discovery

6. Hosting & Deployment

Serverless vs. containers vs. edge vs. VPS?
CI/CD requirements?
Scaling characteristics?

7. Authentication & Authorization

Auth approach: social login, magic links, email/password, SSO?
RBAC needed?
Multi-tenancy considerations?

8. Database & Data Layer

Database type: relational, document, key-value, graph?
Real-time sync, full-text search, vector storage, caching needs?
Read/write ratio?

9. Backend / API Architecture

Monolith or microservices?
REST vs. GraphQL vs. tRPC vs. gRPC?
Background job and queue requirements?

10. Frontend Framework & Rendering

SEO requirements (SSR/static)?

Offline support / PWA?

SPA vs. SSR vs. static vs. hybrid?

11. Third-Party Integrations

External services needed (payments, email, analytics, AI APIs)? Pricing cliffs and rate limits?

Vendor lock-in risk?

Phase 3: Post-Stack Refinement

12. Security Vulnerabilities

Known pitfalls for your stack?

Common misconfigurations?

Dependency risks?

13. File Structure & Project Organization

Standard folder structure for your framework?

Monorepo vs. polyrepo?

Feature/module organization?

14. Naming Conventions & Code Style

Naming patterns for your language/framework?

Linters and formatter configs?

15. Testing Strategy

Unit, integration, e2e, property, and fuzz tools?

Coverage target for the MVP?

Mocking / oracle-diffing patterns?

16. Recommended Tooling & DX

Editor extensions?

CLI tools?

Debugging setup?